

黄佳 - ClaudeCode实践 - 第3章 授人以渔：Skills工程实践

熟能生巧，不如教而不劳。

周一清晨，小冰开启了一个全新的Claude Code会话，着手处理本周的首个任务——为支付模块生成API文档。

她首先花费3 min厘清上下文：团队遵循RESTful规范，文档需要采用OpenAPI 3.0格式，错误码应对标公司内部标准，示例数据必须真实有效，且输出内容需要包含中英双语。在将上周同事撰写的规范文档作为参考投喂给模型后，她才正式指令其开始工作。

上周如此，上上周亦是如此。

每一次开启新对话，都需要将这些前置知识重新“投喂”给Claude。更令人沮丧的是，即使在同一会话中，当任务从“生成API文档”切换到“审查代码质量”时，Claude也无法自动适配相应的工作模式——她不得不额外耗费一轮对话，重新阐明审查优先级、输出格式及严重等级分类。这哪里是在使用智能助手，分明是日复一日地为一名患有“失忆症”的临时工重复进行入职培训。

小雪对此深有同感。在与公司数据分析团队协作时，每次委托Claude进行财务分析，她都不得不重新阐释毛利率公式、行业基准数据以及报告模板格式。这些核心知识恒定不变，但Claude每次都表现得如同初次听闻。

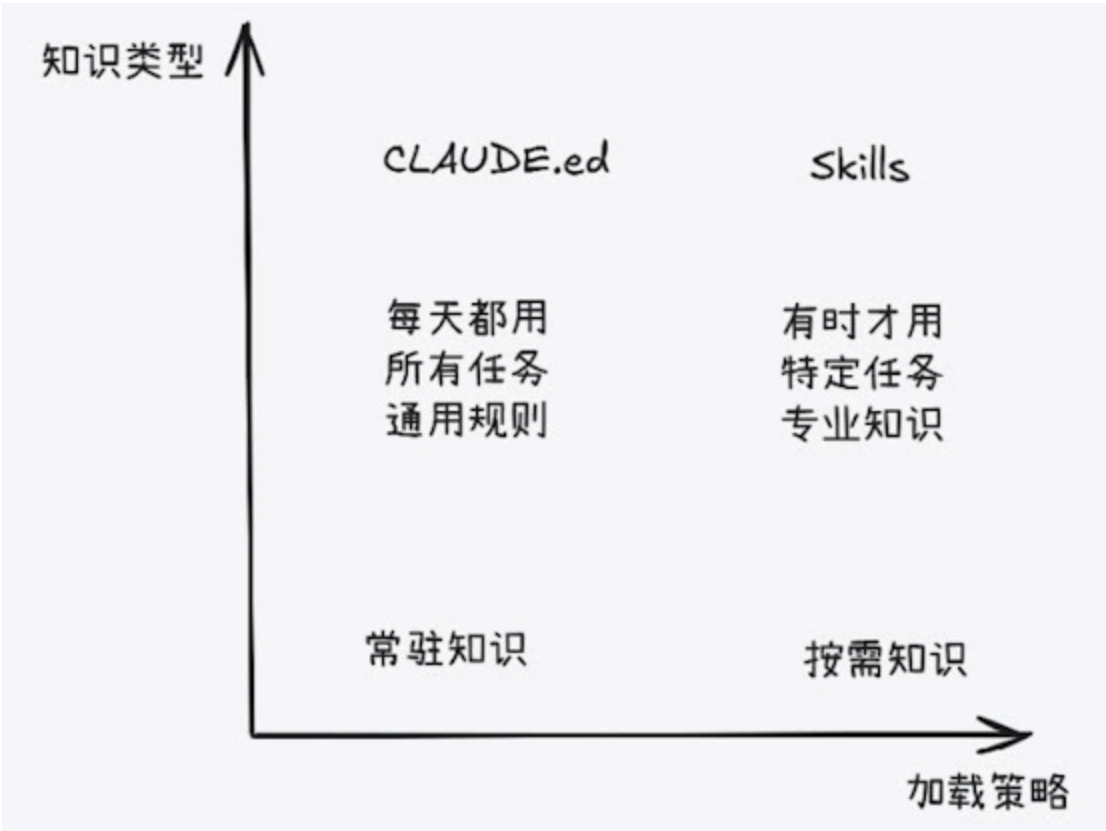
“有没有办法，”她向咖哥问道，“让Claude永久铭记某些特定领域的工作范式？不仅仅是像CLAUDE.md那样项目级别的通用记忆，而是更精准的语境感知，例如，只要我提及‘API文档’，它便能自动知晓执行标准；只要我提及‘财务分析’，它就能自动加载相应的行业基准与报告模板。”

咖哥微微颌首，转身在白板上写下了一个词——Skills。

3.1 从CLAUDE.md到Skills：知识的两个维度

“在深入探讨Skills之前，”咖哥开口道，“我想先厘清一个根本性问题——为何CLAUDE.md已无法满足需求？”

说罢，他在白板上绘制了一个知识类型和加载策略的坐标系（见图3-1）。



(图3-1 CLAUDE.md与Skills知识类型和加载策略)

“CLAUDE.md就像是企业的‘规章制度’，”咖哥解释道，“它定义了项目的基本规则——是用TypeScript还是Python，缩进用Tab键还是空格键，PR标题遵循何种格式。这些规则在每一次对话的每一个瞬间都必须生效，因此它们必须常驻于上下文中，每次全量加载。但这代价显而易见：‘常驻’意味着无论你是否正在处理相关任务，这些Token都会被持续消耗。”

他顿了顿，目光转向小冰：“再想想你的API文档知识——OpenAPI 3.0规范、错误码标准、中英双语格式。这些信息仅在你生成API文档时才被需要。如果强行将它们塞进CLAUDE.md，那么当你调试bug、运行测试或编写配置文件时，这几千Token的文档规范依然会赖在上下文里，白白占据宝贵的空间，稀释了真正关键信息的权重。”

“所以，这就是Skills旨在解决的核心痛点？”小冰若有所思地问道。

“准确地说，Skills解决的是知识的按需投放问题。”咖哥一边说，一边在白板上绘制了一张完整的对比表（见表3-1）。

表3-1 CLAUDE.md与Skills的特征对比

系统文件	承载的内容	生效范围	触发方式	加载策略	典型用途	与Agent的关系	Token消耗	企业本体论
CLAUDE.md	项目通用规范	当前项目	始终生效	每次全量加载	使用TypeScript	所有Agent共享	固定开销	企业规章制度

系统文件	承载的内容	生效范围	触发方式	加载策略	典型用途	与 Agent 的关系	Token 消耗	企业本体论
Skills	专业工作流、领域知识	可跨项目、跨会话	按需激活	渐进式按需加载	如何进行代码审查	可绑定特定 Agent	用时才付按需支付	SOP 操作手册

表3-1最后一列的“企业本体论”引起了小雪的注意。“这是什么意思？”她问道。

咖哥解释道：“一家成熟的企业其实拥有两类截然不同的知识资产。第一类是所有人都必须遵守的通用规则（如考勤制度、安全红线、沟通规范）。这些内容就像贴在办公室墙上的标语，人人可见，时刻生效。这对应着CLAUDE.md。第二类则是特定岗位的标准操作程序(Standard Operating Procedure, SOP)，例如财务如何做月结、运维如何处理故障、客服如何响应投诉等。这些内容不会贴在墙上让所有人看，而是整齐地存放在相应部门的文件柜中，只有当特定岗位的人需要执行特定任务时，才会去翻阅。这，就是Skills。CLAUDE.md是那面墙，而Skills是那个文件柜。你不需要在编写代码时背着财务手册，也不需要在做报表时带着安全规范，只有在‘需要’的那一刻，对应的知识才会被精准提取出来（见图3-2）。”

这个类比精准地锚定了Skills在Claude Code架构中的核心定位。Claude Code对Skills的定义既直白又深刻：**一个Skill是一个包含指令的文件夹，被打包成一个简单的目录结构，用来“教”Claude如何处理特定任务或工作流。**在这个定义中，有两个关键词值得深究。



(图3-2 Skills可按需取用)

第一个是“文件夹”——从“字符串”到“工程化”。Skill绝不仅仅是一段Prompt或一个简单的配置项，它是一个完整的工程化目录——可以容纳复杂的代码库、详尽的领域文档、多样的模板，甚至可执行的脚本。

第二个是“教”——从“约束”到“内化”。定义中使用的是“教”，而非“命令”或“约束”。这微妙地揭示了其运作机制的本质。“教”是一种能力的赋予和内化。通过Skills，Claude真正理解了一个领域的运作逻辑。它不再是被动执行指令的工具，而是变成了该领域的“熟手”。

3.2 解剖一个Skill：骨骼与纹理

“在《庄子·养生主》中，庖丁在解牛时之所以能游刃有余，是因为他早已洞悉牛的骨骼肌理，依乎天理，批大郤，导大窾，而非依靠蛮力去硬砍。”咖哥引用道，“理解Skill的结构，也需要同样的智慧：先看清它的‘骨架’，明白知识是如何安放和组织的，之后才能谈如何精妙地运用它。”

3.2.1 目录结构

咖哥转身在白板上的坐标系旁，列出了一个典型的Skill目录结构。

“这就是‘庖丁解牛’中那只‘牛’的具体构造。”咖哥指着白板上刚刚列出的目录结构，逐一拆解道。

api-doc-generator/	← Skill 名称 (kebab-case, 短横线分隔, 像文件名一样清晰)
├─ SKILL.md	← 【核心骨架】必需的主技能文件 (注意: 大小写必须精确匹配)
├─ scripts/	← 【手脚】可选的可执行脚本, 让Skill能主动“做”事
│ └─ detect-routes.py	← 如自动扫描代码库发现路由
├─ reference/	← 【记忆库】可选的按需加载参考文档, 不占常驻内存
│ └─ openapi-patterns.md	← 只有生成文档时才读取的规范
│ └─ error-codes.md	← 只有处理错误时才查阅的码表
└─ templates/	← 【模具】可选的输出模板, 确保产出格式统一
└─ endpoint-doc.md	← 定义API文档长什么样

“这些看似烦琐的‘清规戒律’，实则是工程实践中用血泪换来的防错机制。”咖哥拿起红笔，在白板上的目录结构旁重重地圈出了几个关键点，神情严肃起来。

文件名：SKILL.md——唯一的“触发器”。 文件名必须采用全大写形式，skill.md、Skill.md、SKILL.MD统统无效。Claude的加载器在扫码文件时，执行的是精确字符串匹配，而非模糊搜索。

目录名：kebab-case——跨平台的“通用语”。 目录名采用全小写+短横线形式（如api-doc-generator），至多64个字符，字符集限制为[a-z0-9-]。不能存在空格、下划线、大写字母、首尾横线和连续横线。这是为了抹平操作系统的差异。Windows操作系统不区分大小写，Linux操作系统严格区分。kebab-case是互联网时代的“世界语”，它保证了你的Skills在任何环境、任何工具链中都能被无损识别。

禁入README.md——纯净的“指令空间”。 Skills目录内严禁放置README.md。人类阅读的文档应放置在父目录中。当Claude激活一个Skill时，它会贪婪地读取目录下的Markdown文件作为上下文。SKILL.md包含的是给Claude看的指令。而README.md通常包含的是给人类看的介绍。如果将README.md混在里面，会干扰Claude对指令内容的理解，引入不必要的噪声。

命名中立：拒绝“claude”或“anthropic”。 name字段中不得包含品牌词。Skills是一种通用的知识封装格式，它的生命力在于开放性。保持品牌中立，是为了让知识本身流动起来，而不是成为

某个产品的附属品。

3.2.2 YAML前置元数据：Skill的“身份证”

SKILL.md顶部的YAML Frontmatter定义了Skill的身份标识与行为边界。完整字段说明如下。

```
---
name: api-doc-generator
  # 【唯一标识符】最多64个字符
  # 若省略，默认使用目录名

description: >
  Generate API documentation from source code. Use when
  the user asks to "write API docs", "document endpoints", or
  "create OpenAPI specs".
  Supports Express, FastAPI, and Spring Boot.
  # 【触发描述】最多1024个字符
  # 若省略，默认使用SKILL.md正文的第一段

argument-hint: "[source directory] [output format]"
  # 【参数提示】在“/”菜单中显示
  # 帮助用户了解该 Skill 接受的输入格式

disable-model-invocation: true
  # 【禁用模型自动调用】设为true时，禁止Claude自动调用
  # 用户必须通过 “/skill-name”手动触发
  # 默认值: false

user-invocable: false
  # 【用户可调性】设为false时，从“/”菜单中隐藏
  # 但Claude仍可自动调用
  # 默认值: true

allowed-tools:
  - Read
  - Grep
  - Glob
  - Write
  - Bash(python:*)
  # 【工具白名单】精确控制Skill执行时可调用的工具及其权限范围

model: haiku
  # 【指定模型】设定该Skill使用的模型
  # 建议简单任务使用Haiku，以提升响应速度并降低成本

context: fork
```

```

# 【执行上下文】设为“fork”时，将在隔离的子智能体中执行
# 确保不污染主对话上下文

agent: Explore
# 【子智能体类型】当context设为“fork”时生效
# 可选值：Explore、Plan、general-purpose或自定义Agent

hooks:
  PreToolUse:
    - matcher: Bash
      hooks:
        - command: echo "$TOOL_INPUT" >> audit.log
# 【生命周期事件钩子】定义Skill激活期间内的事件处理逻辑
# 仅在Skill激活状态下生效
---
```

这些字段可划分为三大逻辑组，分别对应Skills设计的3个核心维度：触发机制、权限控制与运行时环境。

- **身份字段（触发机制）：** 包含name、description、argument-hint。它们定义了“Skill是什么”，负责向Claude和用户清晰传达Skill的功能定位及调用方式，是触发逻辑的基础。
- **权限字段（权限控制）：** 包含disable-model-invocation、user-invocable、allowed-tools、model。它们规定了“谁能调用”以及“能做什么”，通过限制调用来源、可用工具及指定模型，构建起严格的安全与资源边界。
- **执行字段（运行时环境）：** 包含context、agent、hooks。它们决定了“在哪里执行”以及“执行过程中发生什么”，用于配置隔离环境、子智能体类型及生命周期事件钩子，确保任务在预期的上下文中运行。

掌握这3个维度，即掌握了Skills架构设计的完整框架。

咖哥发言

disable-model-invocation 和 user-invocable极易混淆，但它们实际上控制着两个正交的调用方向。**disable-model-invocation: true**的含义是“Claude不能自动触发该Skill，必须由用户通过/skill-name手动执行”。它适用于具有副作用或高风险的操作，如代码提交(/commit)、服务部署(/deploy)等，以确保由用户显式确认。disable-model-invocation的默认值是false。user-invocable: false的含义是“该Skill不会出现在/菜单中，用户无法手动触发，但Claude仍可在内部逻辑中自动调用”。它适用于构建纯知识性或辅助性的参考Skill，隐藏起来可避免菜单杂乱，同时保留Claude在合适时机自动加载使用的能力。user-invocable的默认值是true。默认配置下（即Claude自动调用均为“允许”状态），Skills对Claude和用户双向开放。

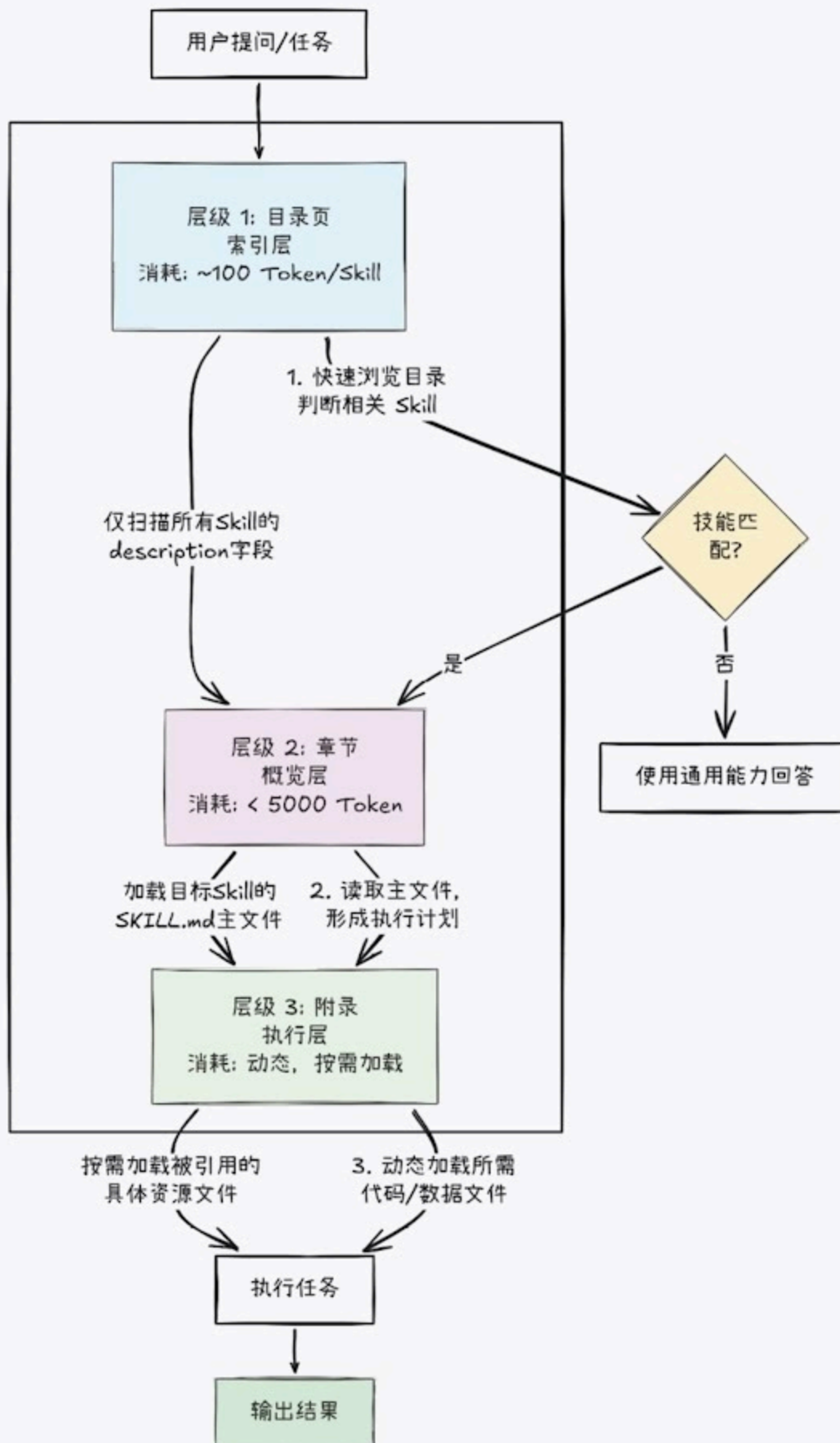
3.3 渐进式披露：知识的投资回报率

“好了，我理解了Skills的结构，”小冰问道，“但如果我有20个Skill，每个都包含数千Token的内容，Claude Code每次对话都要全部加载吗？那和把所有内容塞进CLAUDE.md有什么区别？”

这是一个切中肯綮的问题，也恰恰揭示了Skills系统最巧妙的设计核心——渐进式披露 (Progressive Disclosure)。

3.3.1 图书馆模型

咖哥回答道：“这里可以用一座图书馆来作比。想象你走进一座图书馆寻找资料。你绝不会一次性读完所有藏书，而是遵循一套高效的检索流程——先浏览图书分类编目定位分类，再提取目标书籍查阅图书目录，最后仅精读所需章节。Skills系统正是采用了类似的三层渐进式披露模型（见图 3-3），以极低的Token成本实现海量知识的按需调用。”



(图3-3 三层渐进式披露模型)

该模型在实际应用中带来的性能提升是显著的。以一个完整的财务分析Skill（总计5300 Token）为例，其文件构成如表3-2所示。

表3-2 一个完整的财务分析Skill的文件构成

文件路径	Token数	说明
SKILL.md	800	主逻辑与路由指引
reference/revenue.md	1500	收入数据参考
reference/costs.md	1200	成本结构参考
reference/profitability.md	1000	利润率公式参考
templates/report.md	800	报告模板

不同加载方式的Token消耗对比如表3-3所示。

表3-3 不同加载方式的Token消耗对比

场景	传统全量加载	渐进式披露	节省比例
不激活Skill	0	0	-
扫描阶段(仅判断相关性)	5300	约100	98%
简单请求(仅需要主文件)	5300	800	85%
中等请求(需要一个参考文件)	5300	2300	57%
复杂请求(需要所有资源)	5300	5300	0%

数据指出，绝大多数请求仅需部分资源。当用户询问“毛利率怎么算”时，Claude Code只需要加载SKILL.md (定位路由) 和profitability.md (获取公式)，总计约1800 Token。其他3个文件 (合计3500 Token) 完全保留在磁盘上，零消耗。这是“按需投放知识”的核心经济学优势：用最小的上下文成本，换取最大的知识覆盖范围。

3.3.2 description的预算机制

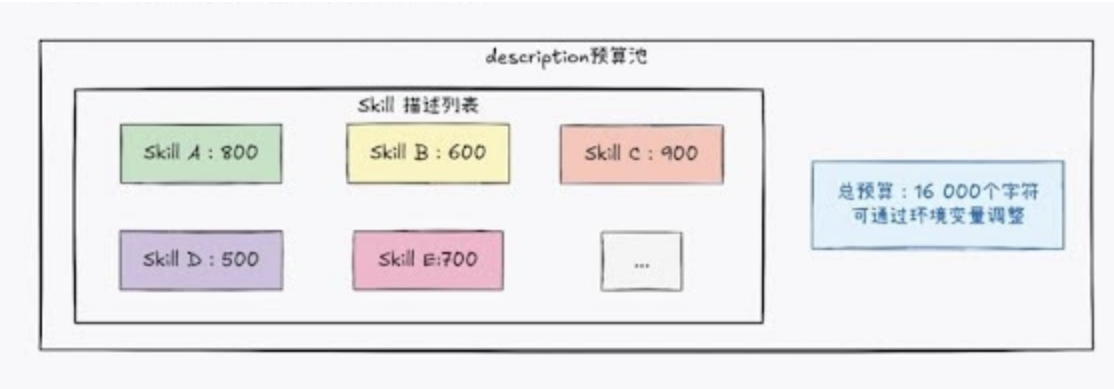
渐进式披露的第一层级，即所有Skill的description（描述），是以常驻方式注入Claude上下文中的。这意味着它们必须共同瓜分一个有限的Token预算，构成了系统的“入口瓶颈”。

Claude Code官方的规则：description总预算上限为上下文窗口总量的2%，若未指定或计算异常，默认固定为16 000个字符。

该预算由所有已安装的Skill平分，而非按需分配。计算公式如下。

单个Skill可用字符数 = 总预算 / Skill的总数

假设你安装了20个Skill，在默认16 000个字符的预算下，每个description平均只能分到800个字符（见图3-4）。



(图3-4 Claude Code的description预算池)

如果某个Skill的description超过800个字符，它将被静默排除。Claude Code完全不知道该Skill的存在，既不会在扫描阶段看到它，更无法在后续步骤中加载它。对用户而言，这个Skill仿佛“消失”了。

咖哥发言

针对description预算的限制，这里有3个关键的实操技巧，能帮你避开陷阱，灵活掌控系统。

- 技巧1，在Skill配置中设置disable-model-invocation: true。该Skill的description不会被注入上下文中。
- 技巧2，运行诊断命令/context可查看是否有超出预算的Skill被“静默排除”。
- 技巧3，通过调整环境变量SLASH_COMMAND_TOOL_CHAR_BUDGET，你可以手动扩大预算池。

这个预算机制揭示了一条核心的工程哲学：**不要盲目创建Skill，而要追求“少而精”的架构设计。**

当你发现需要安装超过20个Skill时，这通常是一个信号，提示你需要重新审视架构策略。

- 合并同类项。将多个零碎的Skill合并为一个综合性的Skill。
- 隐藏内部工具。将纯内部调用的子任务标记为disable-model-invocation: true，减少对预算的占用。
- 采取“Token投资”回报率思维。将description中的每一个字符视为一笔昂贵的“Token投资”——投入精准，就换来高效的检索和准确的执行；投入浪费，不仅浪费自身的预算配额，而且挤占其他Skill的生存空间。

3.4 触发机制：Claude Code如何抉择Skills的调用

Skills的触发机制是整个系统中最关键，也是最需要深入理解的核心环节。它直接决定了Skill能否在恰当时机被激活——这一点甚至比Skill的内容本身的优劣更为重要。毕竟，若无法成功触发，再精彩的内容也将形同虚设。

3.4.1 双通道激活机制

每个Skill均支持两种激活路径。

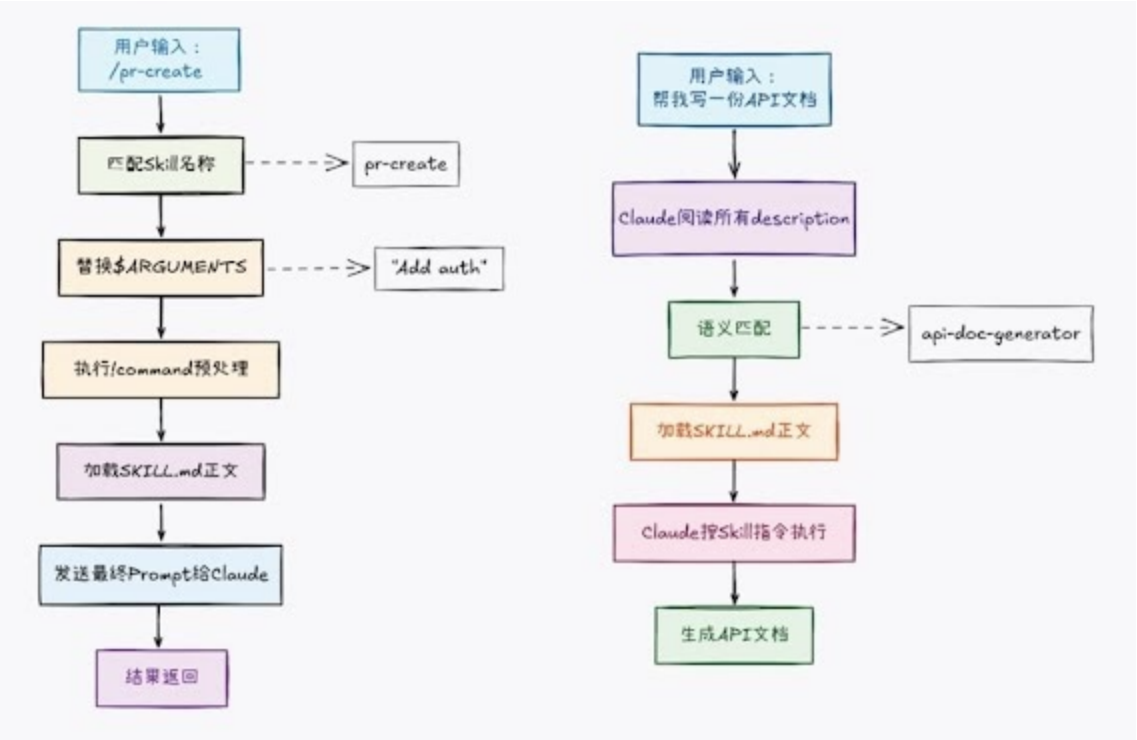
1 显式调用

用户在对话中输入/skill-name，Claude Code即刻加载并执行对应Skill。此方式类似于终端命令，具有明确、直接且无歧义的特点。若Skill定义了argument-hint，用户还可携带参数进行调用，如/commit fix login bug或/review src/auth/login.ts。

2 语义匹配

Claude在深入理解用户意图后，自主研判哪个Skill与当前任务最为契合，从而自动加载。在日常应用中，这正是Skills的核心价值所在——用户仅需要描述需求，Claude便会在幕后智能决策是否调用以及如何调用最合适的Skill。整个过程对用户完全透明，实现了“所想即所得”的无缝体验。

图3-5展示了显式调用的执行流程与语义匹配的执行流程。



(图3-5 显式调用的执行流程（左）与语义匹配的执行流程（右）)

3.4.2 description——Skills的灵魂

语义匹配机制完全依赖于description字段。该字段并非人类阅读的说明性文字，而是Claude在决策“是否调用此Skill”时所依据的**唯一信号**。

Claude Code推荐的description撰写结构公式如下。

[功能定义]（做什么） + [触发场景]（何时用） + [核心能力]（能做什么）

理解上述公式的最好途径，是通过对比“低效”与“高效”的写法。以下展示了两组写法的对比。

```
# 过于模糊，导致Claude无法判断使用时机
```

```
description: Helps with projects.
```

```
# 过于技术化，缺失用户视角的触发关键词
```

```
description: Implements the Project entity model with hierarchical relationships.
```

```
# 仅描述功能，未界定触发场景
```

```
description: Generates API documentation.
```

```
# 正确示范：特点是表述清晰、涵盖具体触发场景、详述核心能力
```

```
description: Generate API documentation from Express, FastAPI, or Spring Boot source code. Use when user asks to "write API docs", "document endpoints", "create OpenAPI specs", or mentions "Swagger". Supports route detection, request/response schema extraction, and authentication requirement marking.
```

description字段设有**1024个字符**的上限。这一空间并不宽裕，因此需要字斟句酌、精心选词。

一个高效实用的写作策略可分为以下3步：

- **第1步，定义核心能力(What)：** 用一句话精准概括该Skill“能做什么”，确定其基本功能定位。
- **第2步，明确触发场景(When)：** 使用Use when user...句式，详细列举各种可能触发该Skill的用户指令、短语或关键词，提高语义匹配的命中率。
- **第3步，划定排除范围(Not For)：** （可选但推荐）如果该Skill容易被误触发，务必加上Not for...，明确指出其不适用的场景，以优化决策的准确性。

咖哥发言

你撰写的description，其核心受众是Claude，而非人类读者。人类阅读文档时倾向于扫描标题、浏览结构，快速抓取大意；而Claude阅读description字段时是在进行深度的语义匹配。它需要捕捉用户意图的细微差别。为了提升触发准确率，你必须在description中穷尽用户可能使用的各种表达方式。例如，针对“生成API文档”这一需求，用户可能会说“生成API文档”“编写接口文档”“输出OpenAPI规范”“帮我写个Swagger”。这些不同的表述都应显式地包含在description中。同义词库越丰富，语义匹配的覆盖面就越广，触发的准确率也就越高。

3.4.3 防止过触发与欠触发

在实践中，Skills的触发机制常面临两种典型的失效模式。理解并修复这些问题，是优化Claude表现的关键。

1 欠触发

现象：Skill本应被调用，但没有被调用。

数据警示：Vercel的评测数据显示，若缺乏明确指引，Agent有56%的概率完全不会去查看可用的Skills。

根本原因：description写得过于技术化或学术化，与用户自然的口语化表述之间存在巨大的语义鸿沟。

修复策略：“翻译”用户语言。在description中大量加入用户常用的表达词汇，涵盖领域术语的同义词、口语化说法，甚至是常见但不准确的表述（例如，很多用户混淆“Swagger”和“OpenAPI”，两者都需要写入）。

2 过触发

现象：Skill在不该被调用的场景下被错误激活。

根本原因：description定义得过于宽泛，包含了太多高频通用词汇，导致匹配阈值过低。

修复策略：引入反向约束。明确划定边界，使用Not for...句式排除干扰项。

示例：“Not for general code questions or debugging. Only for structured documentation generation.”（不适用于通用代码问题或调试，仅用于结构化文档生成。）

要测试一个Skill的触发是否准确，可以构建一个包含10~20个测试用例的验证集，同时覆盖“应触发”和“不应触发”两类场景。

以下是一个实用的测试模板示例。

应该触发的测试用例：

- "帮我生成src/api/的接口文档"

-> 期望触发

- "写一份OpenAPI规范"

-> 期望触发

- "document the REST endpoints"

-> 期望触发

- "帮我写Swagger"

-> 期望触发

不应该触发的测试用例：

- "帮我debug这个API调用"

-> 不应触发

- "这个接口为什么返回500？"

-> 不应触发

- "优化一下这个API的性能"

-> 不应触发

验收标准是：针对相关任务触发率，应达到90%以上，针对无关任务误触发率，应控制在5%以下。

3.4.4 参考型Skill与任务型Skill：两种Skill哲学

在Skill的设计哲学中，disable-model-invocation字段不仅仅是一个简单的开关，它定义了两种截然不同的Skill交互模式——参考型与任务型。

1 参考型Skill

配置：默认行为。

核心逻辑：“按需加载的知识库”。Claude会根据对话上下文自动判断是否需要该类Skill。
description是触发条件，直接注入Claude的上下文中，用于语义匹配。

使用场景：提供知识、规范、框架或标准。用户不需要感知Skill的存在，Claude会在合适时机自动“翻开手册”。

示例：API设计规范、代码审查清单、行业标准文档。

2 任务型Skill

配置：显式设置disable-model-invocation: true。

核心逻辑：“受控的执行工具”。Claude无法自动触发，必须由用户通过/skill-name命令手动调用。
description不注入Claude的上下文，仅作为用户在命令行中选择Skill时的识别说明。

使用场景：具有副作用的操作，需要用户明确授权才能执行。

示例：/commit（提交代码）、/deploy（部署应用）、/notify（发送通知）。

参考型Skill与任务型Skill的对比如表3-4所示。

表3-4 参考型Skill与任务型Skill的对比

维度	参考型Skill	任务型Skill
Claude自动触发	能（基于语义匹配）	不能（必须手动）
用户/触发	能	能
description作用	触发条件（注入Claude上下文）	仅供用户识别（不注入Claude上下文）
消耗Token预算	是（自动加载即消耗）	否（仅手动调用时消耗）
典型场景	知识提供、规范查询	动作执行、副作用操作
经典案例	API规范、审查清单	/commit, /deploy

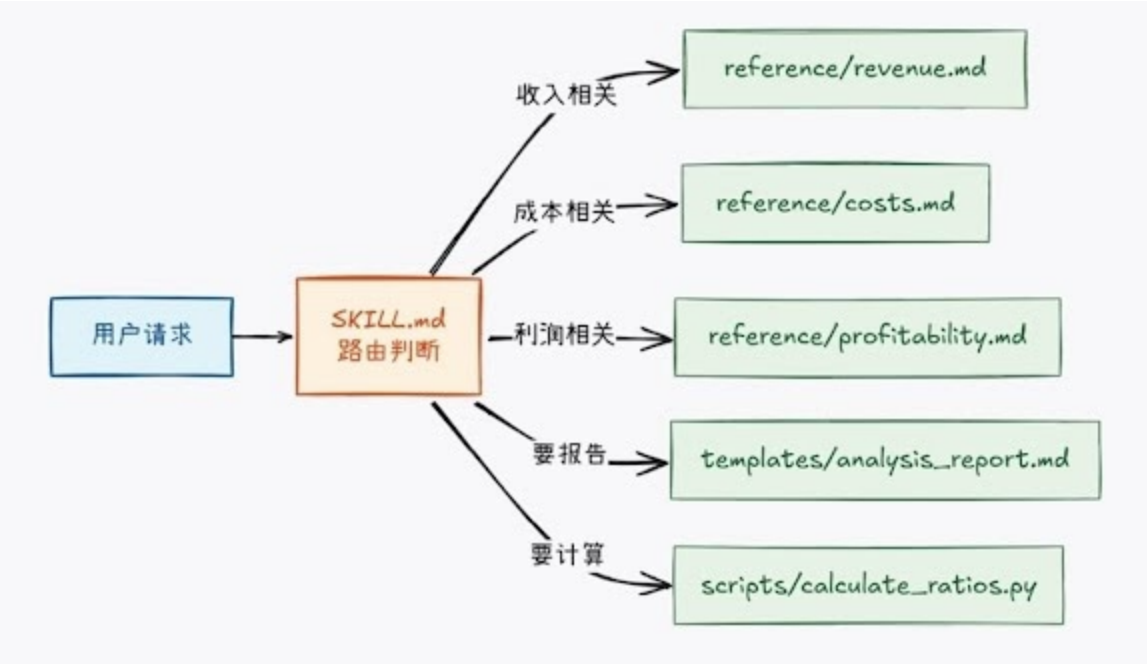
咖哥发言

这里说的副作用(Side Effect)，指的是一个操作不仅返回结果，还改变了系统外部状态。比如提交代码会修改Git仓库历史，部署应用会改变线上服务状态，发送通知会真的给别人发消息。这些动作一旦执行，就会对真实世界产生影响，而且通常不可逆。所以这类Skill通常设计成用户显式触发，而不能让模型自己决定什么时候执行。

小冰问：“一个任务应该设计成参考型Skill还是任务型Skill？”

咖哥答：“请用一个简单的‘最坏情况测试’来做决定：‘如果Claude自动执行这个任务，最坏的情况是什么？’

如果答案让你感到紧张，例如，自动提交了未测试的代码、自动部署了包含bug的版本、自动删除了生产数据、自动发送了错误的通知，必须选择任务型Skill。请加上disable-model-invocation: true。把控制权牢牢握在用户手中，只有当用户显式输入/”



(图3-6 SKILL.md的作用是路由器)

路由设计的核心技巧在于构建“快速参考”(Quick Reference)表格。该表格能以极低的Token（大概3行仅需50 Token），清晰地Claude指引5个关键维度的路由条件。

```
## Quick Reference

| 分析类型 | 触发关键词 | 参考资源 |
|-----|-----|-----|
| 收入分析(Revenue) | 收入、营收、销售额 | `reference/revenue.md` |
| 成本分析(Cost) | 成本、费用、支出 | `reference/costs.md` |
| 盈利分析(Profitability) | 利润、毛利率、净利率 | `reference/profitability.md`
```

相较于让Claude逐行扫描整个SKILL.md以推断“遇到收入问题需要查找revenue.md”，这种结构化表格的效率显著提升，其信息密度可高达前者的10倍。

3.5.2 契约式引用

在SKILL.md中引用辅助文件时，切勿只罗列路径。应当建立一份明确的“契约”，确保Claude清晰知晓3个核心要素：触发时机（何时加载）、资源位置（去哪查找）以及预期产出（获取何物）。

以下示例展示了“弱引用”与“契约式引用”的写法。

```
# 弱引用：缺乏上下文（Claude无法判断何时该加载此文件，缺乏行动指令）
See `reference/revenue.md` for more details.
```

```
# 契约式引用：明确条件+路径+内容预期
## Revenue Analysis
When the user asks about revenue growth, ARPU, or revenue
composition:
→ Load `reference/revenue.md` for calculation formulas and
industry benchmarks.
```

这一设计理念与子智能体流水线中的“交接契约”一脉相承：下游消费者不仅需要知道上游的位置，更必须明确上游能提供什么。

3.5.3 500行法则

Claude Code建议将SKILL.md的篇幅控制在500行以内。

“为何设定为500行？”小冰问道。

咖哥答：“这是因为500行代码等于2000~3000 Token，是单个Skill激活后合理的上下文开销。将其与Claude的System Prompt及当前对话历史累加，能确保总Token数维持在可控范围内。若超过500行，通常意味着你将‘参考资料’与‘路由指令’混淆了——此时的应对策略并非继续扩充内容，而是立即进行重构。”

表3-5列出了当Skills内容过载时的重构信号及对策。

表3-5 Skills内容过载时的重构信号及对策

重构信号	对策
大段公式或规范说明	移至reference/目录
多个完整示例(单个超过30行)	移至examples/目录
多个输出模板	移至templates/目录
可独立执行的逻辑	封装为scripts/脚本
多个平行的功能模块	考虑拆分为多个独立的Skill

3.6 allowed-tools：知识约束行动

allowed-tools 是Skills 安全架构中的核心字段。它不仅仅是一份权限清单，更体现了一项深层设计原则：**知识应当约束行动**。

具体的权限配置应基于Skills业务逻辑的“认知”。

代码审查Skills：审查过程仅需读取代码，严禁修改，因此仅授予只读工具。

文档生成Skills：需要创建新文件，但绝不应篡改既有文件，因此仅授予写入(Write)权限，但明确禁止编辑(Edit)权限。

测试运行Skills：仅需要执行特定的测试命令，因此精确限制可执行的Bash命令，防止任意命令注入。

3.6.1 权限设计模板

以下是针对不同Skill类型的allowed-tools配置最佳实践。这些配置体现了最小权限原则，确保每个Skill仅拥有完成其特定任务所需的最低限度工具权限。

```
# 审计类Skill：严格只读
allowed-tools:
  - Read
  - Grep
  - Glob

# 生成类Skill：可写不可改
allowed-tools:
  - Read
  - Grep
  - Glob
  - Write

# 分析类Skill：只读+特定脚本
allowed-tools:
  - Read
  - Grep
  - Glob
  - Bash(python:*)

# 执行类Skill：受控命令白名单
allowed-tools:
  - Read
  - Bash(git status:*)
  - Bash(git add:*)
  - Bash(git commit:*)
  - Bash(npm test:*)
```

3.6.2 Bash的精细控制语法

Bash工具支持通过前缀匹配机制，实现对可执行命令的细粒度管控。其核心语法为Bash(prefix:)，其中prefix指定允许的命令前缀，作为通配符代表后续参数。

权限控制示例如下。

```
Bash(git:*)          # 允许所有以git开头的子命令
Bash(git log:*)       # 仅允许git log及其参数，禁止git push等危险操作
Bash(npm test:*)      # 仅允许运行测试命令，防止误执行npm install或npm publish
Bash(python:*)        # 允许所有Python脚本执行
Bash(./scripts/*:*)   # 允许执行scripts/目录下的特定脚本
```

当Claude尝试执行Bash命令时，系统会进行前置校验。

- 提取前缀：获取用户请求执行的完整命令字符串。
- 匹配规则：检查该命令是否以配置的prefix开头。

执行决策：若匹配成功，则放行执行；若匹配失败，则直接拒绝，并返回权限错误。

遵循权限最小化原则(Principle of Least Privilege)是构建安全Skills的基石。以下是关于allowed-tools配置的正反案例对比。

```
# 精确授权：只明确列出任务所需的具体命令子集
# 场景：代码提交 Skill
# 策略：仅允许 status、add、commit 3个特定子命令
allowed-tools:
  - Bash(git status:*)
  - Bash(git add:*)
  - Bash(git commit:*)

# 过度授权：使用全局通配符等同放弃所有防线
# 场景：错误的用配置
# 风险：允许执行任意Shell命令
allowed-tools:
  - Bash(*)
```

Bash (*) 在安全性上几乎等价于未设置allowed-tools。这意味着Claude Code获得了宿主机的完整Shell权限。对于像/deploy这样的高危任务型Skill，这种配置是致命的。一旦模型被提示注入攻击劫持，攻击者可利用此权限窃取密钥、删除生产数据或植入后门。

3.7 参数传递与动态注入

Skills不仅是静态指令，更支持运行时参数传递与上下文预注入，使其行为能够根据调用场景进行动态调整。

3.7.1 \$ARGUMENTS和位置参数

当用户通过/skill-name arg1 arg2调用Skills时，可在SKILL.md正文中引用表3-6所示的变量。

表3-6 可在SKILL.md正文中引用的变量及其说明

变量	说明
\$ARGUMENTS	所有参数的完整字符串
\$ARGUMENTS	第一个参数(索引从0开始)
\$ARGUMENTS	第二个参数
\$0, \$1, \$2	位置参数的简写形式

一个引用示例如下。

```
---
name: migrate-component
description: Migrate a component between frameworks
argument-hint: "[component] [from] [to]"
disable-model-invocation: true
---

Migrate the $0 component from $1 to $2.
Preserve all existing behavior and tests.
```

当执行调用/migrate-component SearchBar React Vue时，Claude实际接收到的指令为“Migrate the SearchBar component from React to Vue. Preserve all existing behavior and tests.”。

3.7.2 动态上下文注入

这是Skills 系统中最具威力且独一无二的特性。`!command` 语法允许在将SKILL.md发送给Claude之前，先在Shell环境中执行指定命令，并将命令的输出结果直接内联替换到Prompt中。

一个示例如下。

```
## Current Context (Auto-detected)

Current branch:
!`git branch --show-current`

Recent commits:
!`git log origin/main..HEAD --oneline 2>/dev/null || echo "No commits"`

Files changed:
!`git diff --stat origin/main 2>/dev/null || git diff --stat HEAD~3`
```

当执行/pr-create "Add auth"命令时，Claude实际接收到的已是填充了动态数据的Prompt。

```
## Current Context (Auto-detected)

Current branch:
feature/auth

Recent commits:
a1b2c3d Add JWT middleware
d4e5f6g Add login endpoint

Files changed:
src/auth/middleware.ts | 45 +++
src/auth/login.ts      | 82 +++
2 files changed, 127 insertions(+)
```

这一设计的工程价值巨大。`! command` 特性未使用与使用的对比如表3-7所示。

表3-7 `! command` 特性未使用与使用的对比

维度	未使用! command	使用! command
Claude启动时的上下文	空白，需要多轮对话探索	已预注入关键信息
首次响应的工具调用次数	3~5次 (用于收集信息)	0~1次 (直接执行行动)
Token消耗	高	低
响应速度	慢	快
结果一致性	低 (存在信息遗漏风险)	高 (固定注入相同信息)

3.8 作用域与优先级

咖哥发言 严格界定执行边界。Claude在执行! command 时遵循严格的时序：先替换\$ARGUMENTS变量，再执行Shell命令。这意味着用户的输入内容将被直接拼接进Shell命令中，极易受到Shell注入攻击。因此，任何使用! command 语法的Skills，必须通过allowed-tools配置项严格限制其可执行的命令范围，以构建必要的安全围栏。

Skills文件可部署于不同的层级，每个层级对应特定的生效范围与适用场景，具体如表3-8所示。

表3-8 Skills的作用范围与适用场景

存放位置	生效范围	典型用途
企业配置中心	全员生效	强制执行的企业级开发规范与安全策略
~/.claude/skills//	个人所有项目	个人编码习惯、通用工具集及跨项目辅助脚本

存放位置	生效范围	典型用途
/.claude/skills//	仅限当前项目	项目特有的 workflows、业务逻辑定制及团队协作规范
Plugin 内置资源	Plugin 启用时	社区共享的能力包、特定框架的专用指令集

当多个来源存在同名的 Skill 时，系统将按照从高到低的优先级顺序进行解析与加载。

企业策略 > 个人配置(~/.claude/) > 项目配置(.claude/) > Plugin 内置

这一优先级顺序的设计逻辑严格遵循企业治理架构，确保了从“强制底线”到“个性化偏好”的有序过渡。

- **企业策略（不可逾越的底线）：** 企业级配置拥有最高权限，用于强制执行全局安全与合规策略。例如，安全审计类Skill必须强制检查OWASP Top 10漏洞，此类的关键规则绝不允许被项目级或个人配置覆盖，从而确保组织层面的安全红线不被突破。
- **个人配置（个性化效率工具）：** 位于个人目录的配置旨在满足开发者的个人习惯。例如，用户可以定义/commit命令，使其自动按照conventional commit规范生成提交消息，这种偏好仅作用于个人环境，不影响他人。
- **项目配置（业务场景定制）：** 位于项目根目录的配置，专为特定代码库服务。例如，某个前端项目可定制专属的组件文档生成Skill，以适配其特有的技术栈和文档结构。

将Skill目录(.claude/skills/)纳入项目的Git版本控制，是实现团队知识零成本共享的最优解。

- **即插即用：** 团队新成员克隆代码库后，相关Skill自动生效，无需任何手动安装或配置步骤。
- **同步演进：** 随着项目迭代，Skill库可随代码一同更新，确保团队始终使用最新的工作流规范。

3.9 实战：从零构建3类Skill

理论阐述至此已臻完备，接下来我们将进入核心的工程实战环节。

在本节中，我们将通过3个复杂度递进的完整案例，手把手演示如何构建高价值的Skill。这3个案例并非随意选取，而是精准对应了前面所述的3种典型应用场景。

3.9.1 参考型Skill：代码审查

小冰团队每日需要执行大量代码审查。为此，团队制定了以下内部约定。

优先原则：优先关注安全问题，其次是性能问题，最后才是代码风格。

反馈要求：必须提供具体的修改建议，严禁仅指出问题而不给方案。

分级标注：每个问题均需标注严重等级。

鉴于相关规范散落在不同文档中，新人难以一次性掌握，现整理如下核心指南及目录结构。

目录结构如下。

```
code-reviewing/
├── SKILL.md           # 核心审查流程与标准
└── reference/
    └── severity-guide.md # 详细等级判定标准
```

SKILL.md的核心内容如下。

```
----
name: code-reviewing
description: Performs structured code reviews following team
  standards. Checks security vulnerabilities, performance issues,
  and code quality in priority order. Use when user asks to
  "review code", "do a code review", "check this PR", "audit this
  function", or provides code and asks for feedback.
allowed-tools:
  - Read
  - Grep
  - Glob
----
```

代码审查流程

你是一名资深代码审查员。执行审查时，请严格遵循以下优先层级顺序

第一优先级：安全检查

发现以下安全问题应立即报告：

- SQL注入风险：如直接拼接SQL字符串、未使用参数化查询
- XSS漏洞：未转义的用户输入直接输出至HTML
- 敏感信息硬编码：包括密码、密钥、Token、数据库连接字符串等
- 权限验证缺陷：如缺失认证中间件、存在越权访问逻辑

第二优先级：性能问题

- N+1 查询：循环内频繁调用数据库
- 索引缺失：高频查询字段未建立索引
- 重复计算：循环内存在可提升至循环外的不变量计算
- 内存泄漏风险：如未关闭的连接、持续增长的缓存等

第三优先级：代码质量

- 函数过长：超过50行且无合理理由
- 命名不规范：变量或函数命名含义不清

- 错误处理缺失：空的catch块、异常被静默吞掉
- 代码重复：违反DRY原则

输出格式规范

每个发现的问题必须包含以下4项要素：

- 严重等级：Critical/ Major/ Minor
- 问题描述：具体阐述问题所在
- 文件位置：file_path:line_number
- 修改建议：提供具体的代码修正方案或解决策略

若未发现任何问题，请明确回复“通过审查”，并简述已检查的主要方面

注：详细的等级判断标准请参见`reference/severity-guide.md`。

该Skill的设计聚焦三大核心要点。

- **只读权限保障：** 仅开发Read、Grep、Glob工具，从机制上杜绝审查过程中意外修改代码的风险。
- **优先级排序策略：** 严格遵循“安全>性能>质量”的审查顺序，确保关键隐患优先被发现，避免遗漏。
- **结构化输出规范：** 统一采用“严重等级+问题描述+文件位置+修改建议”的输出格式，保证每次审查结果的一致性和可追溯性。

3.9.2 任务型Skill：智能提交

小冰每日需要频繁提交代码（日均十余次），手动撰写commit message既耗时又容易不规范。为此，我们设计了以下任务型Skill。鉴于该操作具有副作用（直接修改Git历史），系统设定为必须由用户主动触发，严禁自动执行。

```
---
name: committing
description: Quick git commit with auto-generated or specified message
argument-hint: "[optional: commit message]"
disable-model-invocation: true
allowed-tools:
  - Bash(git status:*)
  - Bash(git add:*)
  - Bash(git commit:*)
  - Bash(git diff:*)
model: haiku
---

# Task: Create a git commit
```

```

## Input Handling
If a message is provided: $ARGUMENTS
- Use that as the commit message
If no message is provided:
- Analyze the changes with `git diff --staged` (or `git diff` if nothing staged)
- Generate a concise, meaningful commit message

## Current State (Auto-detected)

Git status:
!`git status --short 2>/dev/null || echo "Not a git repository"`

Staged changes:
!`git diff --staged --stat 2>/dev/null || echo "Nothing staged"`

## Steps
1. Check `git status` to see current state
2. If nothing staged, run `git add .` to stage all changes
3. Review what will be committed with `git diff --staged`
4. Create commit with appropriate message
5. Show brief confirmation

## Commit Message Format
- Start with type: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`
- Be concise but descriptive (max 72 chars for first line)
- Example: `feat: add user authentication with JWT`

## Output
Show a brief confirmation:
✓ Committed: [commit message]
[number] files changed

```

上述Skill展示了多项高级特性的深度组合。

- **安全控制(disable-model-invocation: true):** 强制禁用模型调用，确保执行过程纯粹依赖预设脚本，防止意外的AI推理介入。
- **动态参数(\$ARGUMENTS):** 支持灵活的参数传递机制，允许用户直接指定提交信息或留空以触发自动生成。
- **上下文预注入(! command):** 利用Shell命令在执行前即时捕获并注入当前的Git状态。这使得Claude在启动时即拥有完整的上下文信息，不需要额外调用工具即可感知变更内容，显著提升了响应速度。

- **成本与性能优化(model: haiku):** 指定使用轻量级模型(Haiku)。鉴于提交操作主要依赖规则匹配而非复杂推理，该配置在保证准确性的同时，有效降低了延迟与资源消耗。

一个使用示例如下。

```
# 自动生成提交信息(commit message)
/committing

# 指定提交信息
/committing fix: resolve login validation bug
```

3.9.3 复合型Skill：财务分析 (渐进式披露完整案例)

本案例展示了一种高度模块化的渐进式披露架构。该设计通过分层解耦，实现了复杂任务的高效管理。

- 主控层：由主文件负责意图识别与流程路由。
- 知识层：由参考文件提供垂直领域的深度知识与基准。
- 规范层：由模板确保输出格式的高度一致性。
- 执行层：由脚本封装确定性的计算逻辑，消除幻觉风险。

目录结构如下。

```
financial-analyzing/
├── SKILL.md                # 【主控层】核心入口：负责意图识别与流程路由
├── reference/              # 【知识层】领域知识库
│   ├── revenue.md         # 收入分析：公式定义、行业基准与关键指标
│   ├── costs.md           # 成本分析：成本结构、分摊逻辑与基准
│   └── profitability.md   # 盈利分析：利润率计算模型与评估标准
├── templates/             # 【规范层】输出标准化
│   └── analysis_report.md # 分析报告：结构化模板，确保交付物格式统一
└── scripts/               # 【执行层】确定性计算
    └── calculate_ratios.py # 计算引擎：执行精确的财务比率运算，避免模型计算误差
```

SKILL.md的核心内容如下。

```
---
name: financial-analyzing
description: Analyze financial data, calculate financial ratios,
  and generate analysis reports. Use when the user asks about
  revenue, costs, profits, margins, ROI, financial metrics, or
  needs financial analysis of a company or project.
allowed-tools:
```

- Read
- Grep
- Glob
- Bash(python:*)

Financial Analysis Skill

You are a financial analyst. Help users analyze financial data, calculate key metrics, and generate insightful reports.

Quick Reference

Analysis Type	When to Use	Reference
-----	-----	-----

Revenue Analysis	收入、营收、销售额	`reference/revenue.md`
Cost Analysis	成本、费用、支出	`reference/costs.md`
Profitability	利润、毛利率、净利率	`reference/profitability.md`

Analysis Process

Step 1: Understand the Question

- What financial aspect is the user asking about?
- What data do they have available?

Step 2: Gather Data

- Request necessary financial data from user
- Or read from provided files/sources

Step 3: Calculate Metrics

For specific formulas and calculations:

- Revenue metrics -> see `reference/revenue.md`
- Cost metrics -> see `reference/costs.md`
- Profitability metrics -> see `reference/profitability.md`

To run calculations programmatically:

```
```bash
```

```
python scripts/calculate_ratios.py <data_file>
```

#### ### Step 4: Generate Report

Use the template in `templates/analysis\_report.md` for structured output.

## ## Output Guidelines

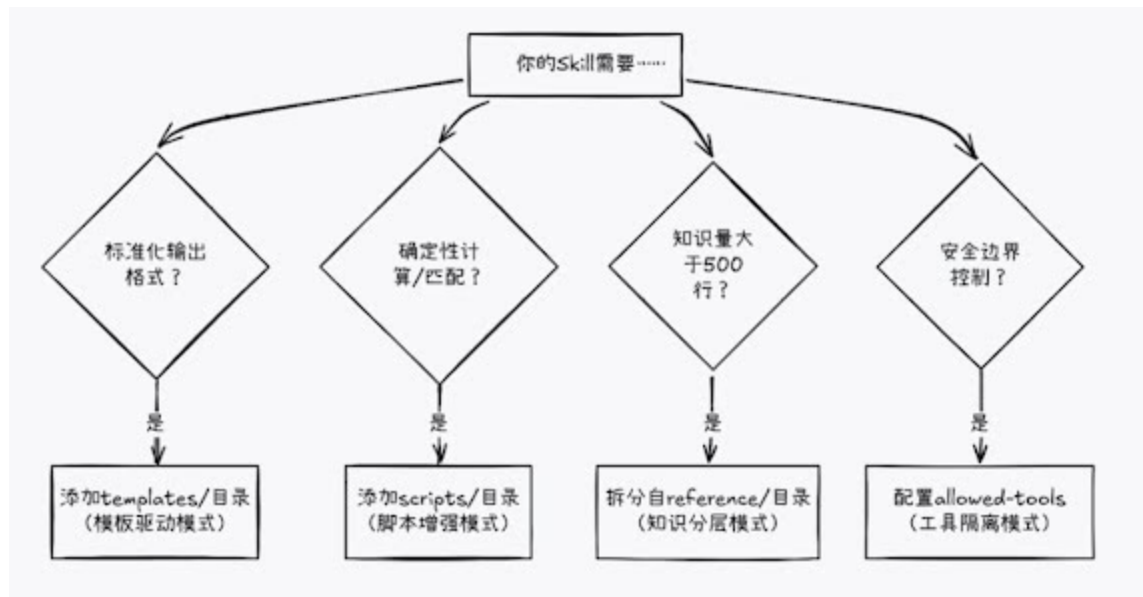
1. Always show your calculations
2. Explain what each metric means in plain language
3. Provide context (industry benchmarks when available)
4. Give actionable recommendations
5. Never make up financial data - ask for clarification if incomplete

上述SKILL.md仅约200行，完全符合“500行法则”。其中的“快速参考”(Quick Reference)表格充当了路由器的角色：当Claude识别到“毛利率”时，能直接定位至reference/profitability.md获取公式，不需要加载收入与成本相关的冗余文件。此外，calculate\_ratios.py脚本封装了确定性计算的逻辑，使Claude不需要执行易出错的浮点数运算，只需要调用脚本即可获得精确结果。

这是“渐进式披露”理念的完整体现：以最小的Token投入，实现任务完成质量的最优化。

## 3.10 Skills的4种设计模式

从工程实践中，我们提炼出4种经过验证的Skill设计模式（见图3-7）。这些模式并非互斥，成熟的Skills系统通常会组合运用多种模式。



(图3-7 Skill设计模式组合决策树)

- **模板驱动模式：** 利用预定义模板严格约束输出格式。该模式适用于周报、事故报告、审查报告等需要标准化输出的场景。在此模式下，Claude的输出将严格遵循模板结构，确保结果具备可预期性、可对比性，并支持自动化后处理。
- **脚本增强模式：** 将确定性计算逻辑封装为脚本，由Claude调用执行而非自行推导。该模式适用于财务计算、正则匹配、数据转换等场景。相较于大模型推理，脚本执行更精准、更节省

Token且更具可复现性。一条经验法则是：如果发现自己的SKILL.md中编写公式使Claude运行计算，请立即停止——该逻辑应当被移至脚本中。

- **知识分层模式：** 依据使用频率对知识进行分层组织。遵循“80/20法则”（即80%的请求仅需要20%的核心内容），将高频知识内联至SKILL.md，而低频知识则置于引用文件中按需加载。这正是“渐进式披露”理念的模式化总结。
- **工具隔离模式：** 通过allowed-tools机制严格界定Skill的能力边界。这属于安全设计范畴，而非单纯的功能设计，其核心价值在于明确“禁止做什么”，这往往比定义“能做什么”更为关键。例如，审计类Skill不授予写入权限，生成类Skill不授予修改权限。

## 3.11 测试与迭代

小冰问：“写完Skill就结束了吗？”

咖哥答：“当然没有。Skill是活文档，需要持续打磨。”

Claude Code官网推荐了3类核心测试方法，以确保Skill的健壮性。

- **触发测试：** 准备10个应触发Skill的问题和10个不应触发的问題，以验证Claude判断的准确率。目标是，相关任务触发率需要高于90%，而无关任务误触发率应低于5%。
- **功能测试：** 验证Skill加载后的执行质量。检查点包括输出格式是否符合预期、检查项是否完整覆盖、边界情况是否得到妥善处理。
- **性能对比：** 针对同一任务，分别在“有Skill”和“无Skill”两种状态下各执行5次，对比Token消耗量、用户修正次数以及最终输出质量。

如果你发现自己反复手动修正Claude的输出（例如，每次都要提醒它“记得标注认证要求”），这是SKILL.md正文需要更新的明确信号。将修正逻辑直接写入SKILL.md，下次就不会发生同类错误。

这个迭代过程与传统软件的bug修复循环完全一致：发现问题→定位原因（description不精确？步骤遗漏？格式定义模糊？）→修复文档→验证效果。

Claude Code还提供了一个名为skill-creator的专用Skill，它能协助你从自然语言描述中生成SKILL.md的初始版本。指令示例如下。

```
Use the skill-creator skill to help me build a skill for [your use case]
```

该工具的存在本身揭示了一个重要事实：Skills系统已经足够成熟，实现了“自举”，即利用Skills系统自身来构建新的Skill。

## 3.12 从软件工程看Skills

正如子智能体一样，Skills也是一个全新的概念。它的核心本质，是将软件工程中历经验证的经典智慧，迁移并适配到AI Agent的知识架构之中。

## 1 关注点分离(Separation of Concerns)

核心理念：“授人以鱼，不如授人以渔”。

Skills将解决问题的方法、步骤与经验沉淀为可复用的知识资产，而非仅提供一次性答案。这使得Agent从依赖“临时灵感”转变为能够稳定复现高质量工作流。

三层架构职责划分如下。

- *CLAUDE.md*：全局规则（项目背景、通用规范）。
- *Skills*：专业工作流（特定领域的复杂逻辑封装）。
- 子智能体：任务执行（动态规划与实时操作）。

工程师警告：严禁将所有逻辑塞入*CLAUDE.md*。如同在软件开发中将所有代码写在main函数中一样，这会导致上下文冗余、维护困难且难以扩展。

## 2 依赖倒置(Dependency Inversion)

核心机制：面向接口编程，而非面向实现编程。

Claude不直接依赖Skill的具体内部实现（如具体的脚本或Prompt细节），而是依赖其抽象接口（即description和输出契约）。只要保持接口契约不变，开发者可以随时替换、重构或升级Skill的内部逻辑。对Claude和用户来说，这种变化是完全透明的，极大地降低了系统的耦合度。

## 3 缓存优化与惰性加载

核心策略：渐进式披露=按需加载。

“渐进式披露”是一种典型的惰性加载(Lazy Loading)策略。系统不在启动时预加载所有知识库，而是在首次需要特定技能时才加载相关资源。

这与Web开发中的代码分割(Code Splitting)和数据库中的延迟加载的思想异曲同工：在正确的时间加载正确的资源，从而最小化初始开销，提升响应速度。

## 4 最小权限原则

安全基石：allowed-tools是安全经典在AI领域的直接映射。

仅授予Skill完成其职责所恰好足够的权限，不多也不少。如同Linux操作系统中使用chmod精确控制文件读写执行权限、AWS中使用IAM Policy精细限定服务访问范围。

明确“不能做什么”比定义“能做什么”更能保障系统安全，防止恶意代码或幻觉导致的越权操作。

## 5 开放标准

生态愿景：声明式、自包含、知识本位。

Anthropic 将 Skills 作为 Agent Skills 的开放标准 (agentskills.io) 规范推广。自2025年12月发布以来，已有超过27个Agent平台（包括OpenAI Codex CLI、Google Gemini CLI、Cursor、GitHub Copilot等）提供原生支持。

Skills成功的三大本质属性如下。

- **声明式(Declarative)**： 纯Markdown格式，任何大模型均可读取和理解，无黑盒二进制。
- **自包含(Self-contained)**： 一个文件夹即包含全部所需，复制即安装，无需复杂的依赖管理。
- **知识本位(Knowledge-Centric)**： 核心价值在于内容本身而非特定格式，不绑定单一平台。

开发者在Claude Code中精心设计的Skill，理论上可以无缝迁移至任何支持该标准的AI平台，真正实现了“一次编写，到处运行”。

这不仅仅是一个关于产品特性的故事，而是一个关于行业标准的故事。你所学到的，不只是“如何在Claude Code中配置某项功能”，而是“如何为AI Agent生态构建标准化的知识包”。

## 本章小结

Skills直击开发者日常痛点：如何让Claude永久铭记专业工作范式，从而避免在每次对话中重新“教学”。在Claude Code的五层架构中，Skills位居知识层——上承工具层（定义“能做什么”），下启智能体层（决定“谁来做”）。它向下通过allowed-tools约束工具行为，向上利用skills字段为子智能体注入专业知识，并与CLAUDE.md形成平行互补之势（前者常驻生效，后者按需调用）。

理解这套机制，关键在于把握四大核心设计。

- **渐进式披露**： 实现知识的按需加载——从始终驻留上下文的description，到触发时加载的SKILL.md正文，再到执行中动态读取的引用文件。这一机制在多数场景下可节省50%~98%的Token消耗。
- **语义触发**： 确保专业能力在恰当时机自动激活。其中，description是此机制的灵魂：它并非供人类阅读的说明书，而是辅助Claude决策的“语义指纹”。
- **安全约束**： 通过allowed-tools将知识限定在安全的行动边界内，实现“知行合一”的安全设计。
- **动态注入**： 借助参数传递(\$ARGUMENTS)与命令执行(! command)，赋予Skill依据运行时上下文灵活应变的能力。

从更宏观的视角审视，Skills实质上是人类组织数十年知识管理经验的数字化映射。在此体系中，SKILL.md等同于部门SOP首页，reference/对应企业知识库，templates/即标准化输出模板，而scripts/则是自动化工具集——这些角色在任一成熟企业中皆能找到精准对标。

更为关键的是，Skills通过开放标准(agentskills.io)规范，将这一映射从企业内部拓展至行业层面：它推动知识管理从企业级的“SOP”跃升为行业级的“ISO标准”，确保Skills的知识价值不再受制于单一平台，而是成为整个生态的通用资产。

在第4章中，我们将深入探讨Claude Code扩展体系中具有里程碑意义的核心特性——子智能体。如果说Skills赋予了Claude专业的“知识”，那么子智能体则赋予了一种更为本质的“能力”：将复杂任务拆解并分派给多位拥有独立上下文的“专家”，让它们在隔离的空间中深度协作，最后仅将精炼的结论汇聚到主对话中。

## 思考题

1. 在日常工作中，有哪些领域知识或工作规范是你需要反复向Claude解释的？不妨尝试将它们提炼并封装为一个Skill的description。在仅1024个字符的限制下，你会优先保留哪些核心信息？
2. 请设计一个名为 `/deploy/` 的任务型 Skill。在设计方案中，需要重点考虑以下要素：  
`disable-model-invocation` 配置的必要性、`allowed-tools` 的精确权限范围、执行 `command` 前需要预加载的上下文（如当前分支、最近构建状态等），以及完善的错误处理路径。最终，请输出完整的 Frontmatter 配置及 `SKILL.md` 骨架。
3. 假设项目中已安装 25 个 Skill，`description` 总字符预算为 16 000。如何在严守预算的前提下，最大化每个 Skill 的触发准确率？哪些类型的 Skill 适合配置 `disable-model-invocation: true`，从而释放宝贵的 `description` 空间？